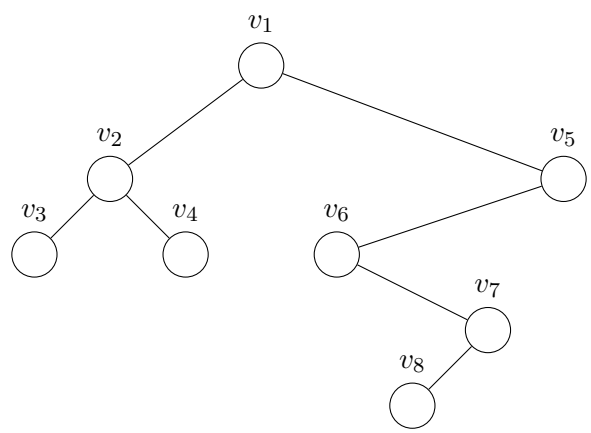
Introduction

Definition 1. A **binary tree** is a tree (a connected acyclic graph) with the following properties

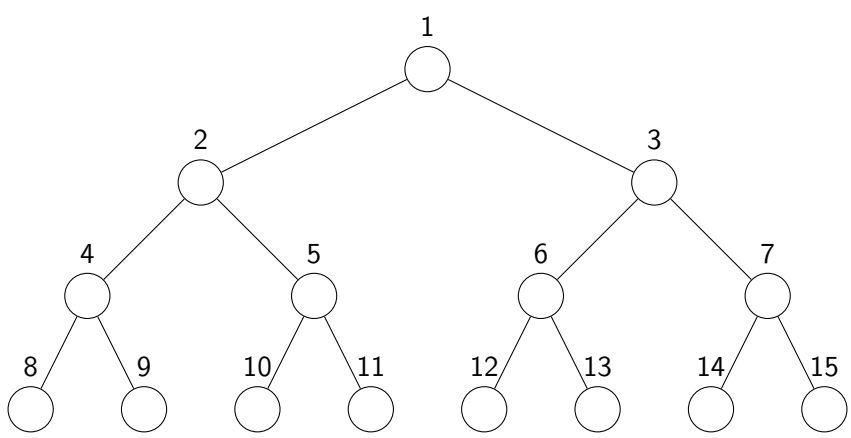
1. Every node has zero, one or two children.
2. One node is the root.
3. Every node, except the root, has one parent.
4. A node with no children is called a leaf.

Definition 2. The **height** of a binary tree is the number of edges on the longest path from the root to a leaf.



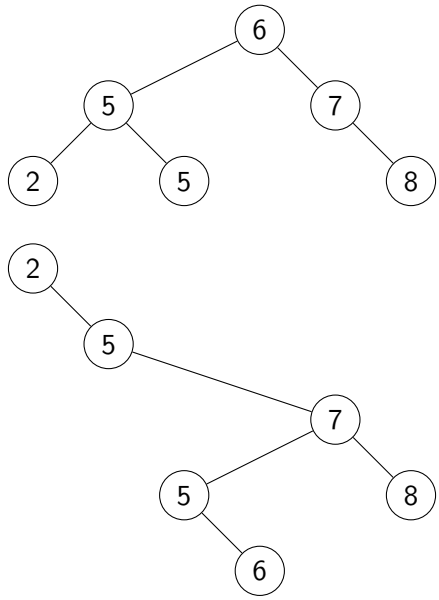
Definition 3. A **complete binary tree** is a binary tree where:

1. All the internal nodes have EXACTLY two children
2. All the leaves are at the same distance from the root.



Definition 4 (Binary Search Tree Property)**.** A binary tree T is a binary search tree if: let node y be a descendant of node x.

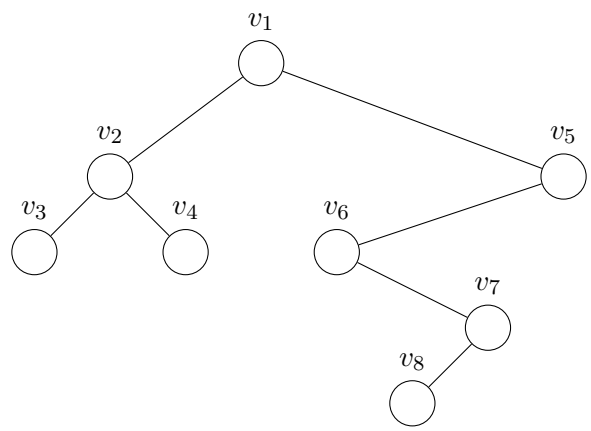
1. If y is in the left subtree of x, then $y.\texttt{key} \leq x.\texttt{key}$
2. If y is in the right subtree of x, then $y.\texttt{key} \geq x.\texttt{key}$.



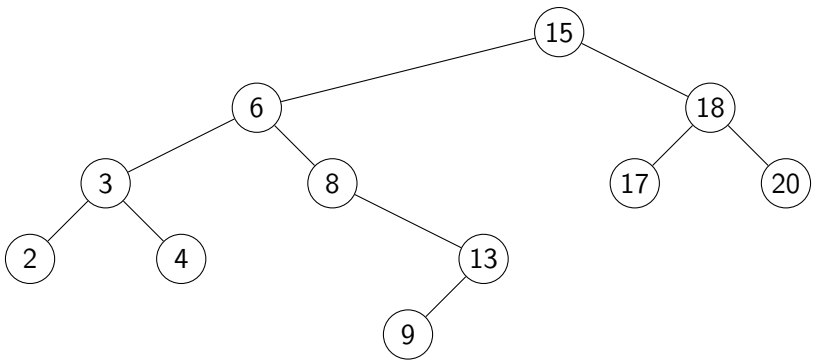
Example 1. Assign the values

$$4, 7, 9, 10, 13, 15, 22, 29$$

to the binary tree and make it a binary search tree.



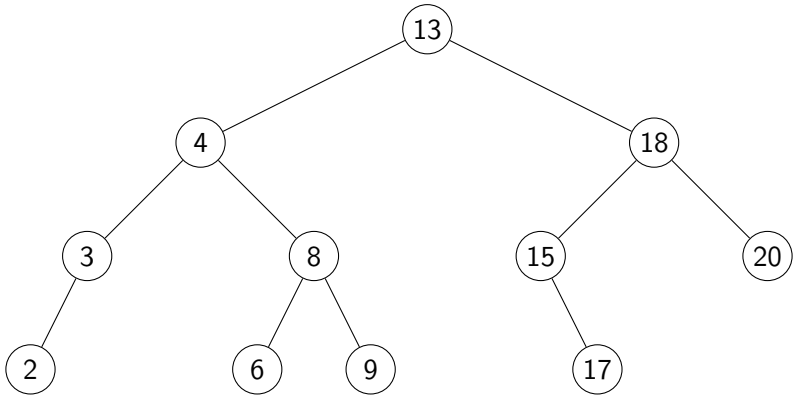

```
function InorderTreeWalk(x)
    if x ≠ NIL then
        InorderTreeWalk(x.left)
        Print(x.key)
        InorderTreeWalk(x.right)
    end if
end function
```



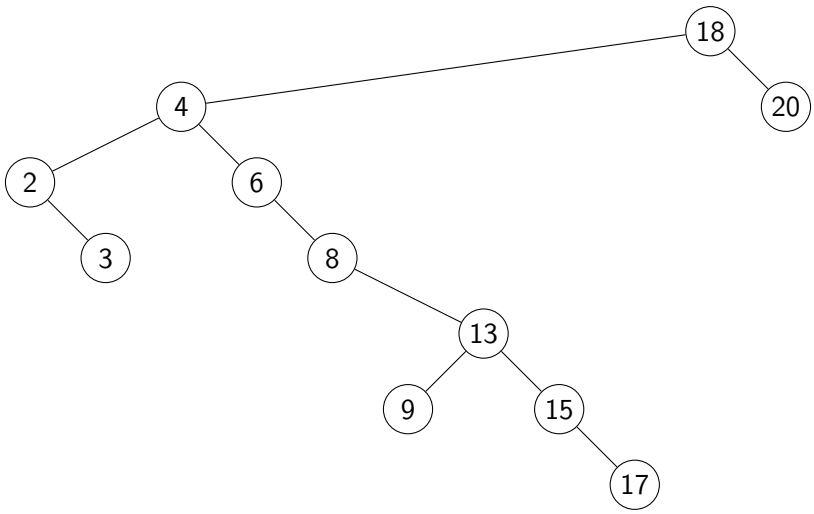

```
function TreeSearch(x,K)
    if (x = NIL) or K = x.key then
        return x
    else if K < x.key then
        return TreeSearch(x.left,K)
    else
        return TreeSearch(x.right,K)
    end if
end function
```

```
function IterativeTreeSearch(x,K)
    while (x ≠ NIL) and (K ≠ x.key) do
        if K ≤ x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return x
end function
```

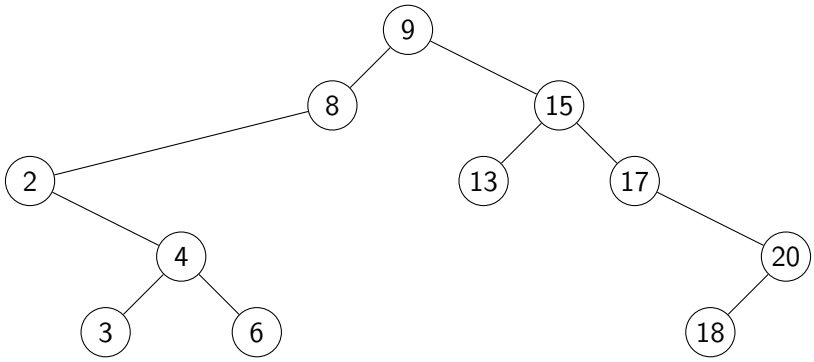
Some trees are "balanced"



and others are not (i.e. they are "unbalanced")

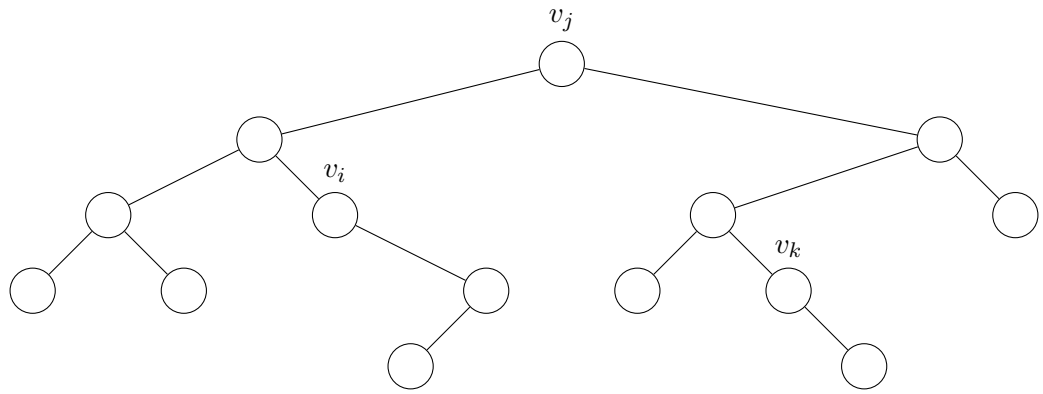


Methods



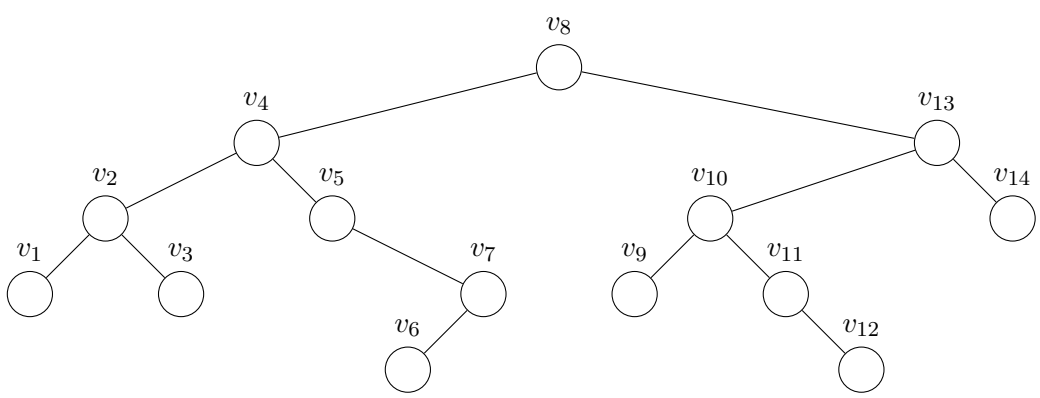

```
function TreeMin(x)
    while x.left ≠ NIL do
        x = x.left
    end while
    return x
end function
```

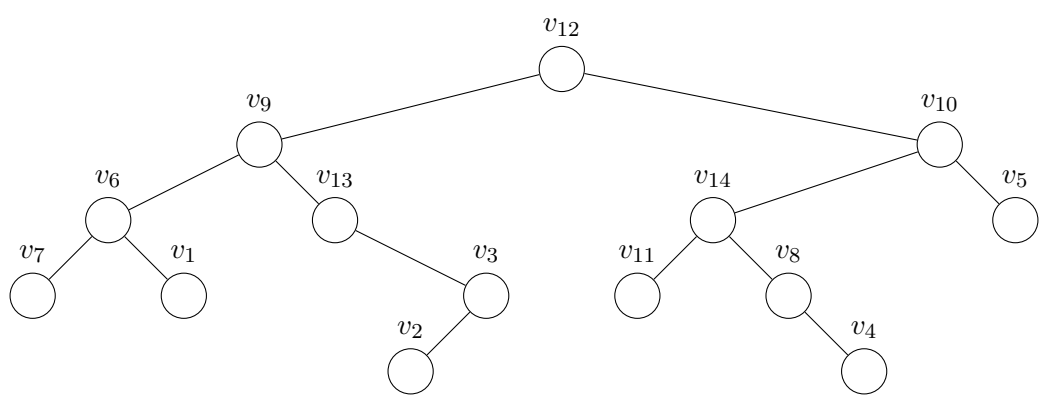
```
function TreeMax(x)
    while x.right ≠ NIL do
        x = x.right
    end while
    return x
end function
```



1. If node v_i is in the subtree rooted at v_j.`left`, then $v_i \preccurlyeq v_j$
2. If node v_k is in the subtree rooted at v_j.`right`, then $v_j \preccurlyeq v_k$

Note: If node $v_i$is in the subtree rooted at v_j.`left` and node v_k is in the subtree rooted at v_j.`right`, then $v_i \preccurlyeq v_j \preccurlyeq v_k$ (i..e $\preccurlyeq$ is a transitive binary relation).
Question: What is the in order sequence of vertices for the following trees?

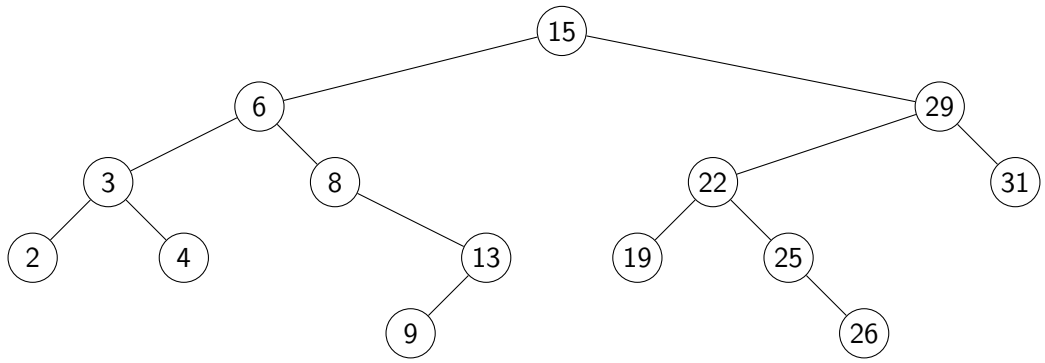




Definition 5. The **successor** of node v_i is the next node in the inorder sequence.

Definition 6. The **predecessor** of node v_i is the previous node in the inorder sequence.

Observation: If T is a binary search tree and all the values at nodes are different, then the successor of node v_i is the node with smallest value greater than the value of v_i.

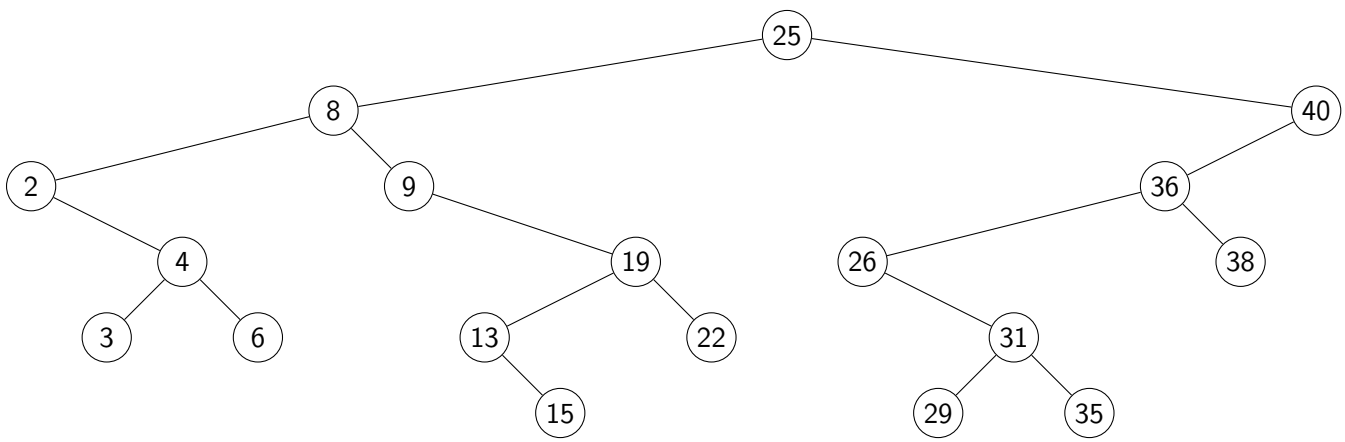


Case I If $x.\texttt{right} \neq$ **NIL**, return $\texttt{TreeMin}(x.\texttt{right})$

Case II If $x.\texttt{right} =$ **NIL**, return closest ancestor y of x where x is in the left subtree of y

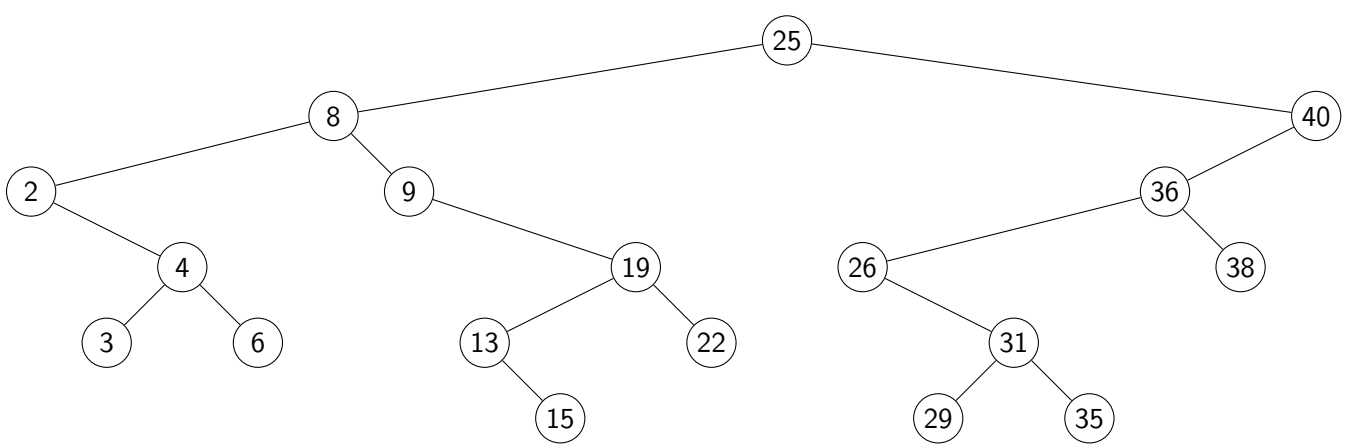
```
function TreeSuccessor(x)
    if x.right ≠ NIL then
        return TreeMin(x.right)
    end if
    y = x.parent
    while (y ≠ NIL) and (x = y.right) do
        x = y
        y = y.parent
    end while
    return y
end function
```

Suppose we want to report all nodes of the following tree with keys in the range $[8, 26]$

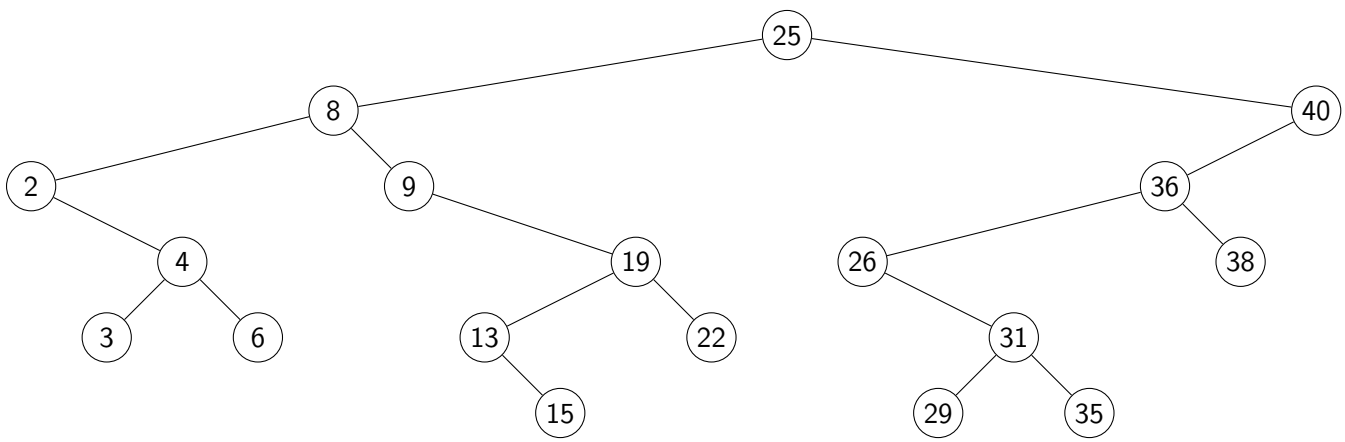



```
function TreeRangeReport(x,k_min,k_max)
    if x ≠ NIL then
        if k_min ≤ x.key then
            TreeRangeReport(x.left,k_min,k_max)
        end if
        if k_min ≤ x.key ≤ k_max then
            Print(x.key)
        end if
        if x.key ≤ k_max then
            TreeRangeReport(x.right,k_min,k_max)
        end if
    end if
end function
```

```
function TreeMinGE(T,K)
  ▷ Return node with minimum key greater than or equal to K
    currentBest = NIL
    v = T.root
    while v ≠ NIL do
        if v.key ≥ K then
            currentBest = v
            v = v.left
        else
            v = v.right
        end if
    end while
    return currentBest
end function
```



What if we only wish to know *how many* nodes lie within a range of values?

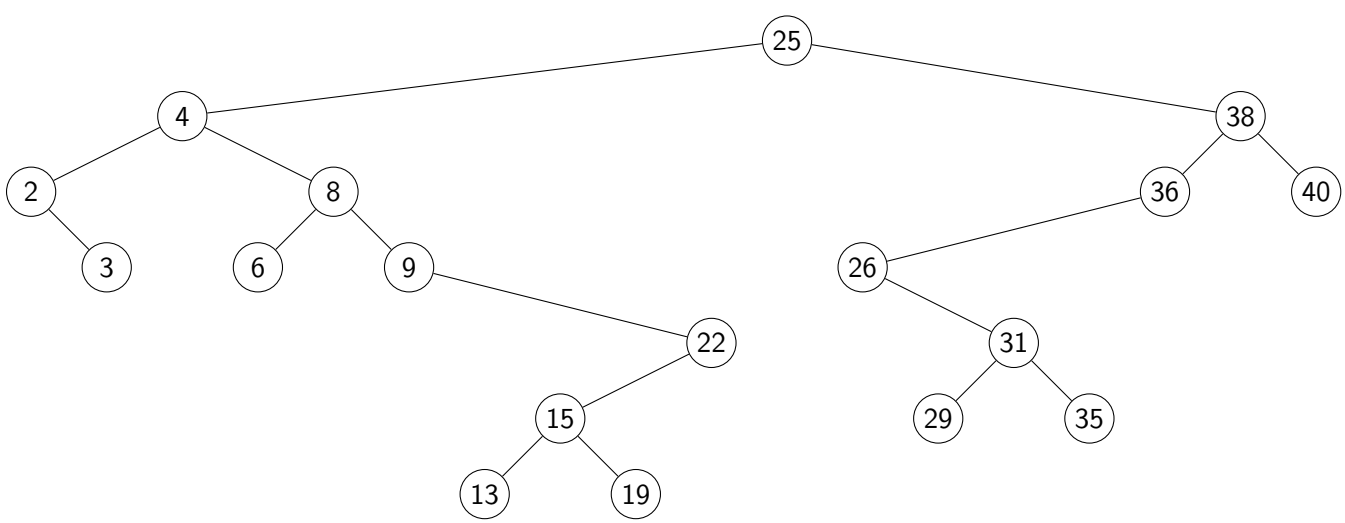



```
function TreeRangeCount(x,k_min,k_max)
    count = 0
    if x ≠ NIL then
        if k_min ≤ x.key then
            count = count+TreeRangeCount(x.left,k_min,k_max)
        end if
        if k_min ≤ x.key ≤ k_max then
            count = count + 1
        end if
        if x.key ≤ k_max then
            count = count+TreeRangeCount(x.right,k_min,k_max)
        end if
    end if
    return count
end function
```

Idea: For each node u in the binary search tree, add a field u.`size` which stores the number of nodes in the subtree rooted at u.

```
function TreeComputeSize(x)
    TreeComputeSize(x.left)
    TreeComputeSize(x.right)
    x.size = 1
    if x.left ≠ NIL then
        x.size = x.size + x.left.size
    end if
    if x.right ≠ NIL then
        x.size = x.size + x.right.size
    end if
end function
```

Compute the size for each node in the tree given below.

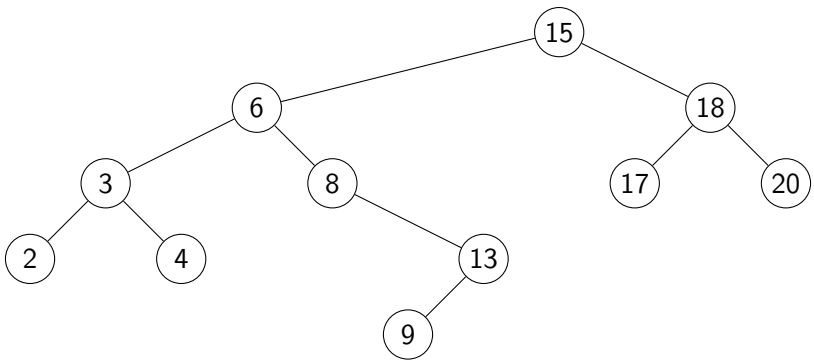


Count the number of nodes greater than or equal to 6.

```
function TreeCountGE(x, K)
    count = 0
    v = x
    while v ≠ NIL do
        if v.key ≥ K then
            count = count + 1
            if v.right ≠ NIL then
                count = count + x.right.size.
            end if
            v = v.left
        else
            v = v.right
        end if
    end while
end function
```

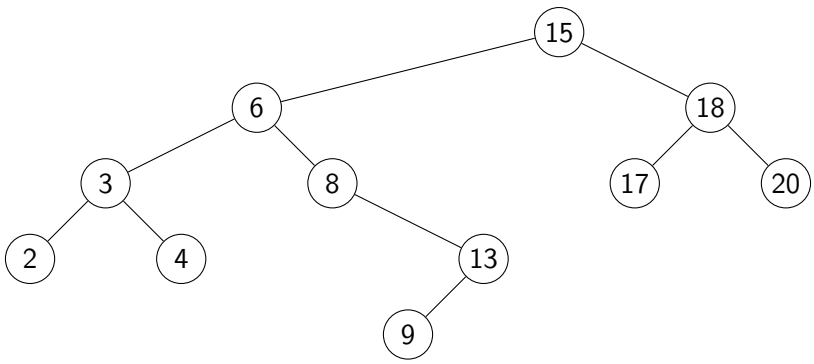
```
function TreeCountLE(x, K)
    count = 0
    v = x
    while v ≠ NIL do
        if v.key ≤ K then
            count = count + 1
            if v.left ≠ NIL then
                count = count + x.left.size.
            end if
            v = v.right
        else
            v = v.left
        end if
    end while
end function
```

```
function TreeRangeCount(x,k_min,k_max)
    v = x
    while (v ≠ NIL) and (x ∉ [k_min, k_max]) do
        if v.key ≤ k_min then
            v = v.right
        else if v.key ≥ k_max then
            v = v.left
        end if
    end while
    if v ≠ NIL then
        count_L = TreeCountGE(v.left, k_min)
        count_R = TreeCountLE(v.right, k_max)
        return 1 + count_L + count_R
    else
        return 0
    end if
end function
```

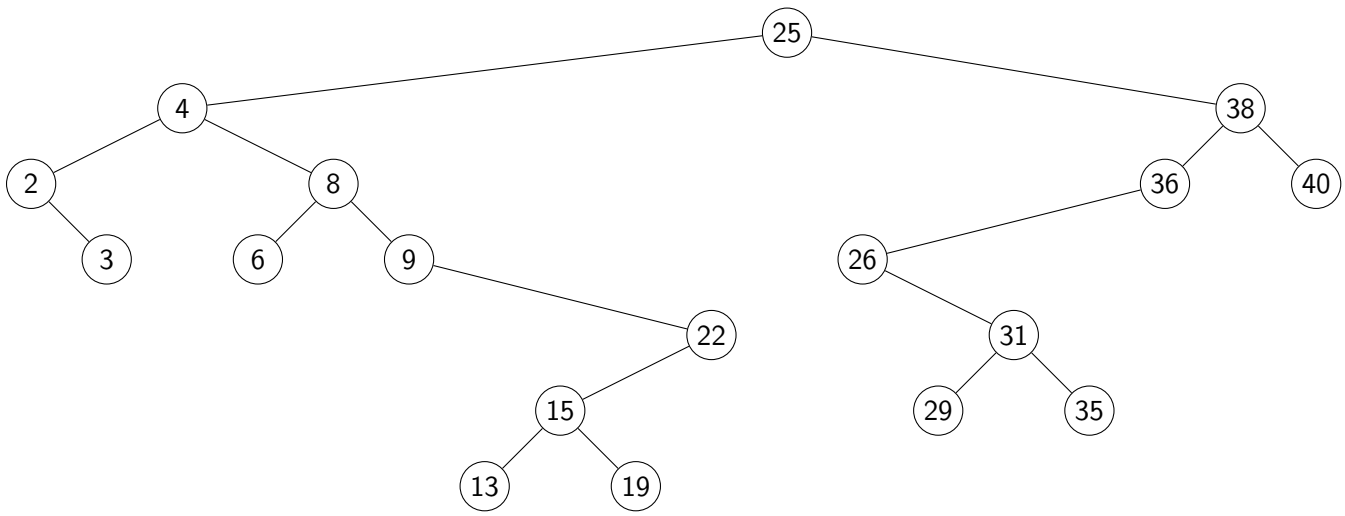



```
function LocateParent(T,z)
    ▷ Return the future parent of z
    y = NIL
    x = T.root
    while x ≠ NIL do
        y = x
        if z.key < x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return y
end function
```

```
function TreeInsert(T,z)
    y = LocateParent(T, z)
    z.parent = y
    if y = NIL then
        T.root = z
    else if z.key < y.key then
        y.left = z
    else
        y.right = z
    end if
end function
```



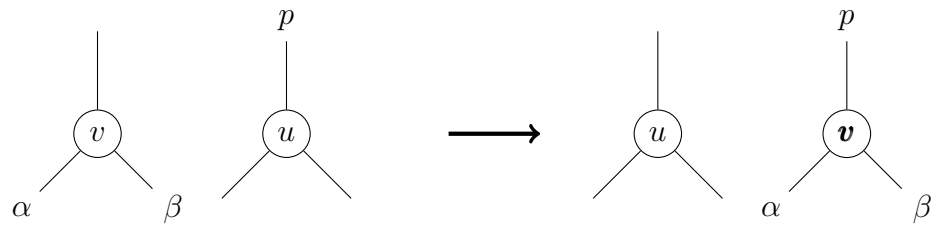

```
function InsertAndUpdateSize(T,z)
    TreeInsert(T,z)
    z.size = 1
    y = z.parent
    while y ≠ NIL do
        y.size = y.size + 1
        y = y.parent
    end while
end function
```



Case I If node z has no children, delete z

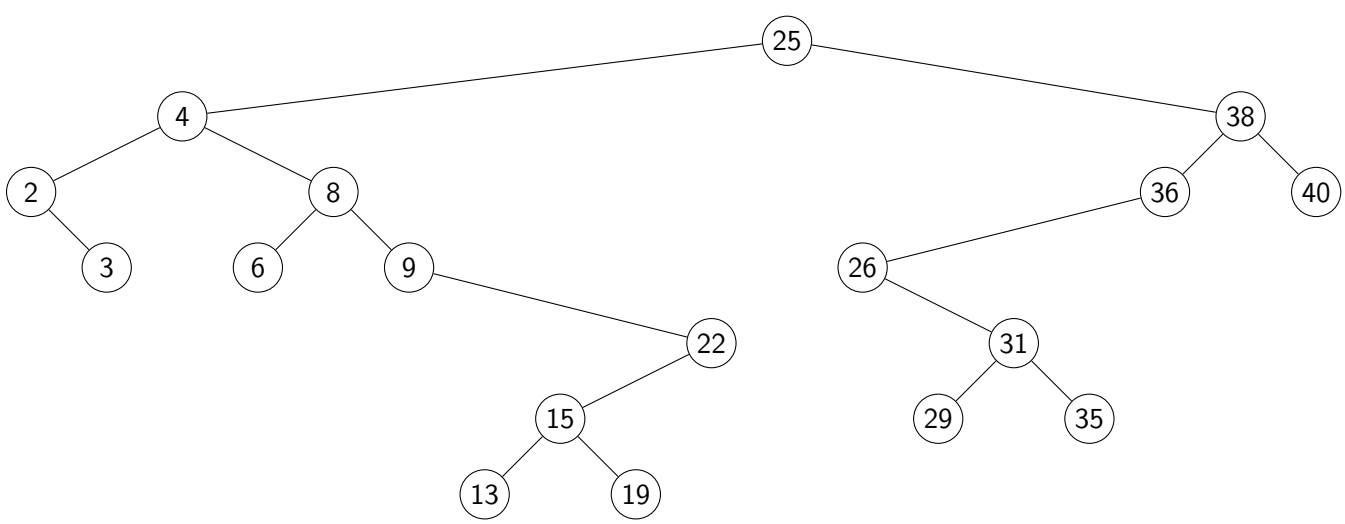
Case II If node z has one child, replace z with its child.

Case II If node z has two children, replace z with its successor y and replace y with its right child.




```
function Transplant(T,u. v)
    p = u.parent
    if p = NIL then
        T.root = v
    else if u = p.left then
        p.left = v
    else
        p.right = v
    end if
    if v ≠ NIL then
        v.parent = p
    end if
end function
```

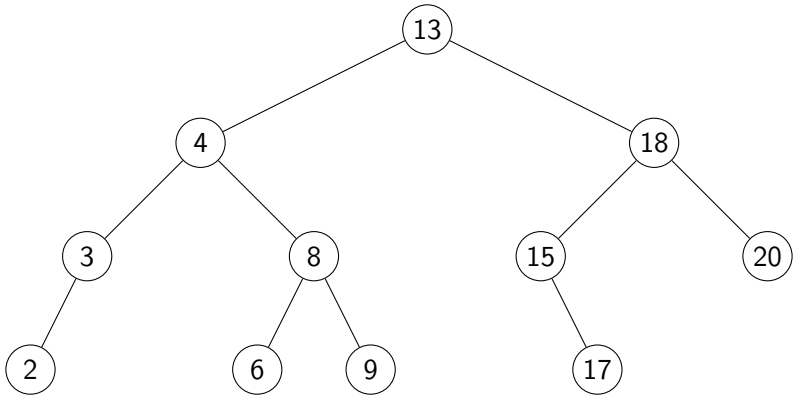
```
function TreeDelete(T,z)
    if z.left = NIL then
        Transplant(T,z,z.right)
    else if z.right = NIL then
        Transplant(T,z,z.left)
    else
        y = TreeMin(z.right) ▷ y is the successor of z
        if y ≠ z.right then
            Transplant(T,y,y.right)
            y.right = z.right
            y.right.parent = y
        end if
        Transplant(T,z,y)
        y.left = z.left
        y.left.parent = y
    end if
end function
```

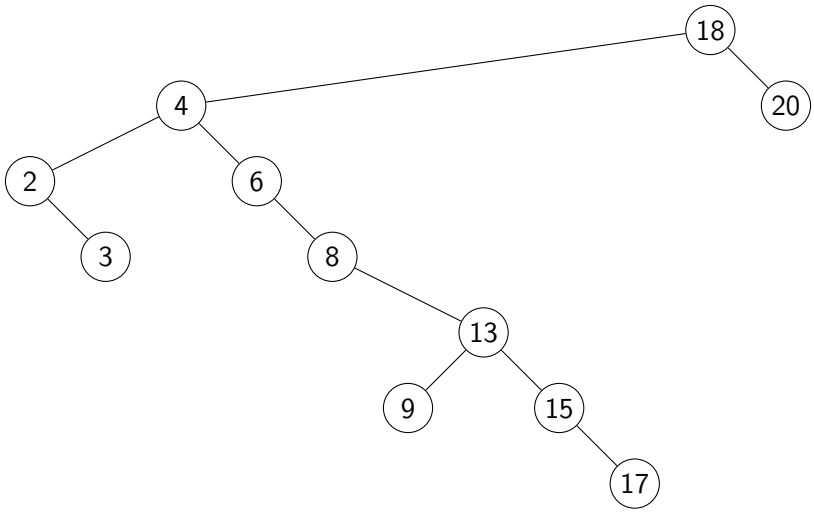


```
function UpdateSize(T,z)
    while z ≠ NIL do
        z.size = 1
        if z.left ≠ NIL then
            z.size = z.size + z.left.size
        end if
        if z.right ≠ NIL then
            z.size = z.size + z.right.size
        end if
        z = z.parent
    end while
end function
```

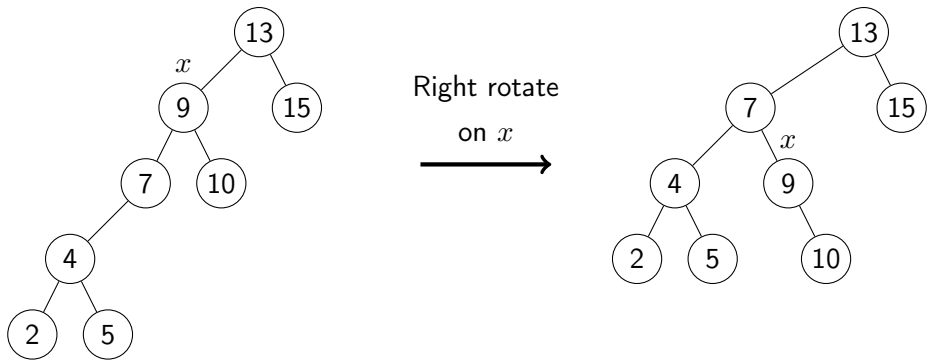
```
function TreeDelete(T,z)
    if z.left = NIL then
        Transplant(T,z,z.right)
        UpdateSize(T,z.parent)
    else if z.right = NIL then
        Transplant(T,z,z.left)
        UpdateSize(T,z.parent)
    else
        y = TreeMin(z.right) ▷ y is the successor of z
        x = y.parent
        if y ≠ z.right then
            Transplant(T,y,y.right)
            y.right = z.right
            y.right.parent = y
        end if
        Transplant(T,z,y)
        y.left = z.left
        y.left.parent = y
        UpdateSize(T,x)
    end if
end function
```

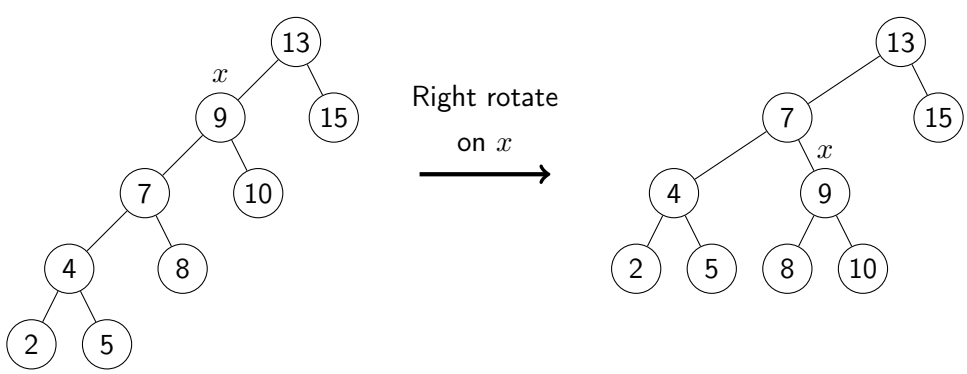
Consider the two binary search trees

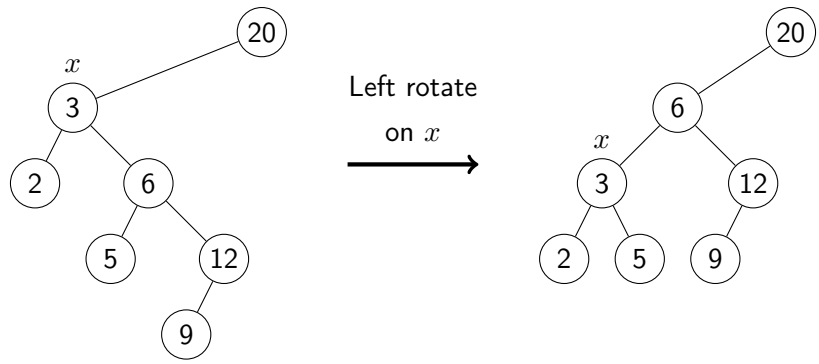


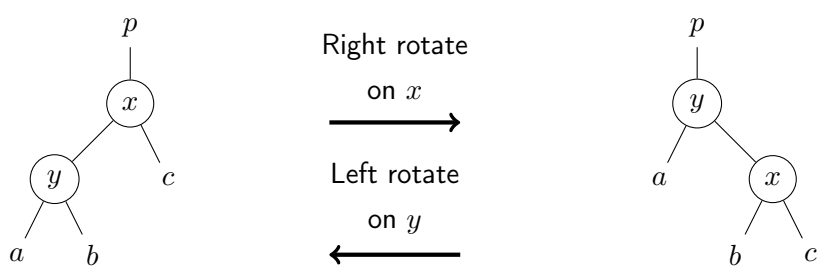


Can we perform any set of operations to transform the unbalanced tree into a more balanced tree?

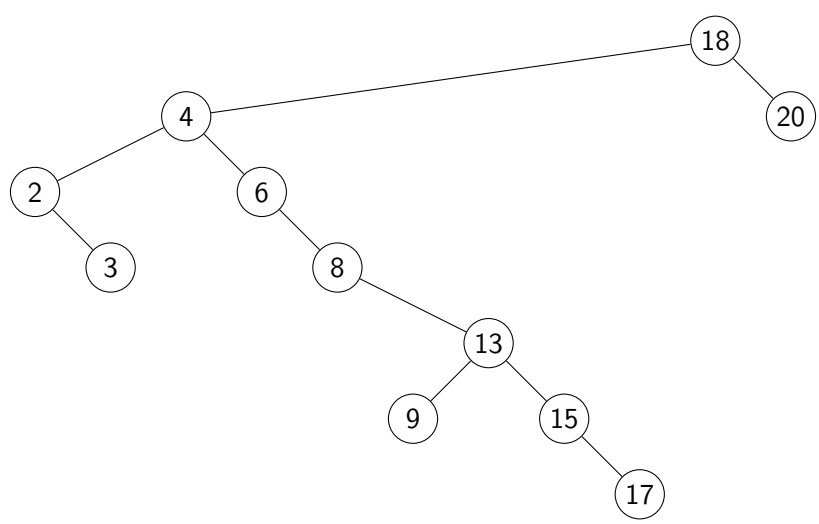

13
x
9
15
7
10
4
2
5
Right rotate
on x
13
7
15
x
4
9
2
5
10


13
x
9
15
7
10
4
8
2
5
Right rotate
on x
13
7
15
x
4
9
2
5
8
10


20
x
3
2
6
5
12
9
Left rotate
on x
20
6
x
3
12
2
5
9


p
x
y
c
a
b
Right rotate
on x
Left rotate
on y
p
y
a
x
b
c

```
function RightRotate(T,x)
    y = x.left
    b = y.right
    Transplant(T,x,y)
    x.left = b
    if b ≠ NIL then
        b.parent = x
    end if
    y.right = x
    x.parent = y
end function
```

```
function LeftRotate(T,x)
    y = x.right
    b = y.left
    Transplant(T,x,y)
    x.right = b
    if b ≠ NIL then
        b.parent = x
    end if
    y.left = x
    x.parent = y
end function
```

Apply a rotation and reduce the height of the tree